
Blackcoin

Quantum Quasar

Protocol V4

ML-DSA Spending Paths, Migration, and Participation Rules

Technical White Paper

Quantum Quasar Developers

Version 30.1.1 - July 2026

<https://github.com/Blackcoin-Dev/Blackcoin>

Contents

Abstract	4
1. Reward and Activity Rules	4
2. The V4 Timeline: Four Phases	5
Exact schedule (mainnet)	5
3. Post-Quantum Cryptography in Blackcoin	6
3.1 Legacy elliptic-curve exposure	6
3.2 ML-DSA-44 signature verification	6
3.3 New witness versions and address types	6
3.4 Self-test on startup	7
4. The Gold Rush Reward Epoch	7
4.1 The whitelist snapshot	7
4.2 The reward schedule	7
4.3 The PoS / PoW split	8
4.4 Qualifying and claiming: QQSIGNAL and QQSPROOF	8
5. Quantum Migration and the Legacy Lockout	9
5.1 Legacy output exposure	9
5.2 The migration path	9
5.3 Final Lockout and legacy spending	10
6. Demurrage and Liveness	10
6.1 Exactly which coins are subject	10
6.2 The inactivity clock and the grace period	10
6.3 The decay curve	11
6.4 Keeping eligible holdings at 100%	11
7. Quantum Staking: Tiered, Cold, and Pooled	12
7.1 Tiered self-staking	12
7.2 Cold staking: separate the owner key from the staking key	12
7.3 Operator bonds and the per-pool cap	13
7.4 Autonomous redelegation	13
8. Reserved v15 EUTXO Design and RGB	13
8.1 EUTXO witness v15 is frozen in v30.1.1	13
8.2 RGB, client-side fixed-supply assets	14

9. Wallet and RPC Reference	14
9.1 Chain and schedule	14
9.2 Staking, Gold Rush, and mining	14
9.3 Quantum addresses and migration	15
9.4 Quantum staking, cold staking, operator bonds	15
9.5 Demurrage	16
9.6 EUTXO inspection and RGB	16
9.7 Wallet maintenance	16
9.8 The GUI	17
10. Worked Wallet Examples	17
Example A, A legacy holder preparing to migrate	17
Example B, The active staker during Gold Rush	18
Example C, The forgotten wallet	18
Example D, Running a staking pool	18
11. Economic Incentives and Participation Conditions	19
12. Security Considerations	19
Appendix A: Consensus Constant Reference	20
Appendix B: Glossary	23

Abstract

Blackcoin launched in 2014 as a Proof-of-Stake (PoS) cryptocurrency. Its legacy spending paths use elliptic-curve signatures, which are exposed to a sufficiently capable quantum computer when the relevant public key is available. PoS block production also depends on holders operating eligible staking nodes.

Quantum Quasar (Protocol V4) introduces the NIST-standardized ML-DSA-44 signature scheme as a consensus-enforced spending path, a scheduled migration from legacy elliptic-curve outputs to ML-DSA-protected outputs, and rules for Gold Rush rewards, liveness demurrage, quantum staking, and Final Lockout.

This paper describes the V4 phase schedule, reward formulas, wallet workflows, RPCs, and listed consensus constants, with source references. The numbers and source references describe the v30.1.1 release source.

1. Reward and Activity Rules

V4 conditions Gold Rush rewards on qualifying PoS or PoW activity. It also applies an inactivity schedule to eligible quantum holdings after Migration and closes the legacy ECDSA spending path at Final Lockout. The rules below define those conditions and their phase boundaries.

- **Gold Rush rewards.** Eligible PoS participants must solve and signal within the required window; PoW participants must submit valid claims. Merely holding coins or keeping a node online does not guarantee a reward.
- **Liveness demurrage.** After the migration era, eligible quantum holdings that remain inactive for more than six months begin the specified decay. Decayed principal is permanently burned when spent; it is never added to transaction fees or paid to a miner or staker. For an eligible direct or tiered v16 holding, the wallet can attempt a low-fee liveness attestation when staking is enabled, the wallet is normally unlocked, and a safe fee input is available. A cold-stake output is also subject to the activity clock; a successful coinstake spends and recreates it, resetting that clock.
- **Scheduled quantum migration.** Legacy elliptic-curve outputs are the network's quantum attack surface. Gold Rush keeps ordinary quantum funding disabled so the base chain remains legacy-compatible. It is followed by an **18-month Migration phase** in which holders can move eligible legacy coins into ML-DSA-protected addresses with `migratetoquantum`, subject to wallet, backup, fee, and confirmation conditions. Final Lockout then closes the legacy spending path. Legacy value stays spendable for the full Gold Rush-plus-Migration schedule, but the migration transaction itself must be made during Migration.

These rules make Gold Rush rewards conditional on qualifying activity and set a height boundary for legacy ECDSA spending.

2. The V4 Timeline: Four Phases

Protocol V4 uses a height-authoritative lifecycle on Blackcoin mainnet. MTP anchors remain as nominal forecasts and for isolated compatibility tests, but timestamp movement cannot advance, delay, skip, or reverse a mainnet phase boundary.

Phase	Mainnet heights	Target duration	Legacy spend?	v14/v16 funding and spending?	Gold Rush rewards?
Legacy	through 5,949,999	-	Yes	No	No
Gold Rush	5,950,000-6,192,999	243,000 blocks (~180 days)	Yes	No; shadow credits are locked	Yes
Migration	6,193,000-6,921,999	729,000 blocks (~540 days)	Yes	Yes	No
Final Lockout	from 6,922,000	permanent	No	Yes	No

The quantum column refers only to authenticated witness-v14 and witness-v16 paths. Witness v15 has no supported funding or spending workflow in any v30.1.1 phase; consensus rejects v15 outputs and spends from Migration onward.

Exact schedule (mainnet)

The consensus boundaries are defined in `src/shadow_schedule.cpp`, `src/shadow.h`, `src/consensus/params.h`, and `src/kernel/chainparams.cpp`.

- **V4 and Gold Rush begin:** height **5,950,000**.
- **Last Gold Rush block:** height **6,192,999**. Migration begins at **6,193,000**.
- **Last Migration block:** height **6,921,999**. Final Lockout and automatic demurrage begin together at **6,922,000**.

The retained time anchors are non-authoritative mainnet forecasts:

- `QUANTUM_QUASAR_MAINNET_V4_TIME = 1783835299` is **2026-07-12 05:48:19 UTC**.
- Adding the 180-day nominal Gold Rush duration gives `1799387299`, **2027-01-08 05:48:19 UTC**.
- Adding the 540-day nominal Migration duration gives `1846043299`, **2028-07-01 05:48:19 UTC**.

Phase membership is computed by `GetQuantumLifecycleState(nTime, nHeight)` (`src/consensus/params.h`). Because Blackcoin mainnet targets a **64-second block time**:

- **1,350 blocks/day, 40,500 blocks/month** (30-day month), **~493,000 blocks/year**.

Legacy elliptic-curve coins remain spendable for Gold Rush and Migration, approximately 720 target days in total. Ordinary quantum funding is deliberately off during the first 180 target days, so holders perform `migrate to quantum` during the following 540-day Migration phase. Final Lockout closes the legacy path at height 6,922,000.

3. Post-Quantum Cryptography in Blackcoin

3.1 Legacy elliptic-curve exposure

Blackcoin's legacy outputs are protected by ECDSA over secp256k1. A sufficiently large quantum computer running Shor's algorithm can recover a private key from a public key. Outputs with exposed public keys, including P2PK coinbase outputs, would be exposed to such a machine. Staking publishes public keys used by those outputs.

3.2 ML-DSA-44 signature verification

V4 introduces **ML-DSA-44** (Module-Lattice Digital Signature Algorithm, the NIST FIPS 204 standardization of CRYSTALS-Dilithium at security level 2) as a native, consensus-verified signature scheme, provided by liboqs 0.15.0 and wrapped in `src/crypto/mldsa.h`. ML-DSA's security rests on the hardness of module lattice problems, which are not known to be broken by quantum algorithms.

Quantity	Size
Public key	1,312 bytes
Secret key	2,560 bytes
Signature	2,420 bytes

These are larger than a 33-byte ECDSA public key and an approximately 72-byte signature. V4 places ML-DSA data in the witness and uses commitment-based addresses; the public key is revealed at spend time.

3.3 New witness versions and address types

V4 defines three new SegWit witness versions. Direct v14/v16 programs and the reserved v15 shape are 32 bytes; tiered v14/v16 programs are 40 bytes (`src/consensus/quantum_witness.h`, `src/addresstype.h`). All use the mainnet Bech32 human-readable prefix `blk` (`src/kernel/chainparams.cpp`).

Witness v	Program	Purpose
v16	32-byte direct or 40-byte tiered program	Quantum migration / tiered staking output: an ML-DSA-protected home for migrated coins. Subject to demurrage.
v15	32-byte commitment	Reserved EUTXO shape : disabled/frozen in v30.1.1 because it has no quantum ownership authorization.
v14	32-byte direct or 40-byte tiered program	Quantum cold-stake : owner/staker-separated delegation output, subject to inactivity demurrage.

For authenticated v14 and v16 paths, the base program carries a 32-byte commitment. A tiered program prepends eight bytes of state to a 32-byte commitment. The bulky ML-DSA public key and signature are supplied in the witness at spend time. The reserved v15 commitment does not commit to an ML-DSA owner; this is why v30.1.1 must not fund or spend it.

3.4 Self-test on startup

The node performs an ML-DSA Known-Answer-Test at startup to check that liboqs is linked and produces the expected signature result before participating in consensus. A build that cannot reproduce the KAT refuses to run.

4. The Gold Rush Reward Epoch

Gold Rush is a six-month bonus-emission epoch beginning with V4. Its PoS and PoW pools credit qualifying staking activity and valid mining claims under the rules below. Holding a balance alone does not produce a credit.

4.1 The whitelist snapshot

At height **5,945,000** (`SHADOW_WHITELIST_HEIGHT`, `src/shadow_schedule.cpp`), the node builds a **deterministic snapshot** of every script holding at least **10,000 BLK aggregate** (`SHADOW_WHITELIST_MIN_BALANCE`, `src/shadow.h`). The snapshot is computed once from the UTXO set at that exact height and is read-only thereafter, so every node derives an identical whitelist.

- Balances are aggregated **per canonical script**. P2PK stake scripts are folded to their P2PKH identity via `CanonicalizeLegacyStakeScript()` so that a holder who staked with raw-pubkey outputs and a holder who used address outputs are treated as one account.
- The snapshot happens **5,000 blocks (~ 3.7 days) before** Gold Rush rewards begin at height 5,950,000, fixing the eligibility set before rewards begin.

10,000 BLK threshold. A canonical target at or above this amount in the snapshot can qualify for Gold Rush PoS credits. The target must also meet the staking and signalling requirements; snapshot membership alone pays nothing.

4.2 The reward schedule

Each Gold Rush block accrues a base reward, `ShadowBaseReward(height)` (`src/shadow.cpp`), over the window from height 5,950,000 to **6,192,999** (`SHADOW_REWARD_END_HEIGHT`), a span of `SHADOW_GOLD_RUSH_BLOCKS = (180 x 24 x 60 x 60) / 64 = 243,000` blocks.

Phase 1 (heights 5,950,000-6,187,599): a halving curve.

$$\text{reward} = (580 \text{ BLK}) \gg (\text{blocks_since_start} / 43,200)$$

The reward starts at **580 BLK/block** and halves every `SHADOW_HALVING_INTERVAL_BLOCKS = 43,200` blocks (~ 30 days):

Month	Height range	Reward/block
1	5,950,000-5,993,199	580 BLK
2	5,993,200-6,036,399	290 BLK
3	6,036,400-6,079,599	145 BLK
4	6,079,600-6,122,799	72 BLK

Month	Height range	Reward/block
5	6,122,800-6,165,999	36 BLK
6 (part)	6,166,000-6,187,599	18 BLK

Phase 2 (heights 6,187,600-6,192,999): a fixed tail.

```
reward = 463 BLK/block
```

The final ~5,400 blocks pay a flat **463 BLK/block**. The two-phase shape is calibrated so that total Gold Rush accrual lands at the hard cap `SHADOW_MAX_EMISSION = 51,437,700 BLK` (`src/shadow.h`); the pool cannot over-issue beyond this cap regardless of block timing.

4.3 The PoS / PoW split

Each block's base reward is divided **evenly** between two independent reward pools (`src/shadow.cpp`):

```
pos_pool_reward = reward - reward/2    (50%)
pow_pool_reward = reward/2             (50%)
```

- **The Proof-of-Stake half** rewards eligible native stakers that meet the solve and signal requirements.
- **The Proof-of-Work half** opens a parallel, opt-in participation lane using a specified Argon2id puzzle (`SHADOW_ARGON2_TIME_COST = 1`, `SHADOW_ARGON2_MEMORY_KIB = 1024` (1 MiB), `SHADOW_ARGON2_LANES = 1`). A valid claim is required for a reward.

Rewards accumulate into pools and are drawn by **claims** under the PoS and PoW eligibility rules below.

4.4 Qualifying and claiming: QQSIGNAL and QQSPROOF

Gold Rush participation is expressed through two on-chain, fee-paying control transactions carried in `OP_RETURN` outputs:

- **QQSIGNAL (Proof-of-Stake side).** A whitelisted holder who has produced a recent PoS block signals eligibility by broadcasting a QQSIGNAL that references their recent solve. "Recent" is defined by the solver-activity window `SHADOW_SOLVER_ACTIVITY_SECONDS = 14 days` (`SHADOW_SOLVER_ACTIVITY_WINDOW = 18,900 blocks`, `src/shadow.h`). A qualifying solve and timely valid signal are both required for PoS credit; a snapshot balance alone is insufficient.
- **QQSPROOF (Proof-of-Work side).** A miner grinds an Argon2id proof (magic `QQSPROOF`, `src/shadow.cpp`) against the target difficulty, a 12-bit base, ASERT- retargeted every 64 blocks within a [10-bit, 18-bit] band, and submits it in a claim transaction that is validated before mempool acceptance. Already-mined blocks through 5,993,199 retain the v30.1.0 first-valid-claim allocation. At the first scheduled halving, height 5,993,200, v30.1.1 begins the QQP3 canonical rank-v1 rule, ranking competing candidates independently of transaction order and evaluating at most 64 Argon2 proofs. QQP3 binds each new proof to its intended height and parent and permits fee-only reimbursement for 64 later blocks on the same branch. The lowest-ranked valid current-origin claim receives the fixed PoW pool after every current loser and eligible late claimant is reimbursed its actual base fee, capped at 0.01 BLK. A late-only block leaves the unreimbursed pool accumulated. At most 0.63 BLK can be reimbursed alongside a winner. Invalid, malformed, expired, off-branch, and excess claims receive nothing.

QQP4 additionally binds the exact single legacy fee-input output. It has a separate consensus activation and is disabled on mainnet in v30.1.1. Readiness or version-bit signalling cannot activate QQP4. Any future QQP4 release must publish an explicit activation height and a tested transition for QQP3 claims that are still inside their

late-inclusion window.

The wallet can automate both when the corresponding staking/mining mode and signing prerequisites are satisfied. See §9 for the exact RPCs (`sendshadowsignal`, `sendshadowpowclaim`, `setpowmining`, `getgoldrushinfo`).

A wallet may have up to 64 independent live `QQSPROOF` claims in its local mempool. A wallet-authored claim that leaves the mempool is different: another peer may retain and later confirm it, so the wallet quarantines the claim, reserves its fee input, and refuses generic abandonment. The built-in miner pauses on actionable or indeterminate quarantined components rather than treating every historical descendant as a separate recovery action.

The Issue #37 wallet release groups wallet-known sibling conflicts and descendants at their nearest current confirmed anchor. One shared engine serves manual GUI, headless, and optional automatic recovery. It separates read-only preview, explicitly acknowledged exact-plan signing and durable draft persistence, and separate commit-and-broadcast authority. Automatic recovery is wallet-scoped, bounded, and off by default; it neither unlocks the wallet nor enables mining.

Either the claim or the resolution may confirm. Only the confirming transaction pays its base-chain fee, and a resolution fee receives no shadow reimbursement. Peers retaining the original may reject the conflict, broadcast does not guarantee confirmation, and the input remains reserved until an active-chain confirmation resolves it. A reorg triggers reclassification, and original-claim confirmation can expose a later confirmed frontier that requires a fresh plan. The full operator model is specified in Gold Rush PoW claim lifecycle and recovery.

5. Quantum Migration and the Legacy Lockout

5.1 Legacy output exposure

Legacy ECDSA spending paths remain exposed when their public keys are available to a sufficiently capable quantum computer. V4 sets a finite Migration phase and then rejects legacy ECDSA spends at Final Lockout.

5.2 The migration path

During Migration, `migratetoquantum` (see §9) can sweep eligible spendable legacy outputs to a **wallet-backed witness-v16 migration** address. When the RPC creates a new ML-DSA key, it writes the key to the wallet database before constructing the migration transaction and refuses the action if storage is not confirmed. The caller must account for unlock, fee, backup, and confirmation conditions.

Critical backup note. ML-DSA keys are *not* derived from the wallet's HD seed. After creating a migration address you **must back up the wallet again**, or a restore from an older backup will not recover the migrated funds. The wallet and this paper both flag this during the wallet workflow.

During Gold Rush, wallets can create and back up quantum addresses and can dry-run migration planning with an existing wallet-backed address. They cannot fund or spend ordinary v14/v16 outputs. `migratetoquantum` becomes actionable at Migration height 6,193,000 and remains available through height 6,921,999. Gold Rush reward credits are separate authenticated synthetic outputs that remain phase-locked until Gold Rush ends and normal maturity is satisfied.

5.3 Final Lockout and legacy spending

At Final Lockout, **height 6,922,000**, the consensus rule `IsQuantumFinalLockout(nTime, nHeight)` (`src/consensus/params.h`, enforced in `src/validation.cpp`) becomes true, and the script flag `SCRIPT_VERIFY_LEGACY_ECDSA_LOCKOUT` (`src/script/interpreter.cpp`) causes **all legacy ECDSA-signed spends to be permanently rejected** with `legacy-spend-disabled`. Exact authenticated v14 and v16 paths remain enabled. Witness-v15 funding and spending remain rejected with the dedicated EUTXO-disabled rules.

The scheduled transition provides about six months of Gold Rush preparation followed by an 18-month Migration phase. The exact height deadline is visible in the wallet, network status RPCs, and this document. Holders must migrate eligible legacy outputs during Migration to retain a spendable path after Final Lockout. The rule rejects legacy ECDSA spending after the deadline; it does not move dormant outputs into ML-DSA-protected outputs.

6. Demurrage and Liveness

Demurrage applies to eligible quantum outputs after more than **six months** of inactivity. A qualifying attestation for a direct or tiered v16 key, or a spend that recreates the output, refreshes the applicable activity clock. Delegation alone does not.

6.1 Exactly which coins are subject

Evaluated per output by `EvaluateDemurrage` (`src/consensus/demurrage.cpp`). The classification, in order:

1. **Demurrage not yet active** -> exempt. Demurrage cannot begin before the migration era (mainnet's height-authoritative schedule ends Migration at block 6,921,999 and activates Final Lockout and demurrage at block 6,922,000).
2. **Explicitly configured exempt scripts** -> exempt. Mainnet currently configures none.
3. **Non-quantum (legacy and everything else) outputs** -> exempt (`"non_quantum"`). Legacy coins are governed by the lockout, not demurrage.
4. **Quantum migration/tiered (v16) and cold-stake (v14) outputs** are subject, but only if inactive beyond the grace period. Eligible direct/tiered v16 keys can use attestations; cold-stake state refreshes through a successful spend/recreation.
5. **Historical v15-shaped outputs** can be classified by the accounting evaluator, but v30.1.1 independently rejects their funding and spending. They have no supported liveness or recreation path, and their metadata is inspection-only.

Thus cold delegation is not a permanent-value shelter. A cold-stake output that never successfully stakes or moves follows the same inactivity curve.

6.2 The inactivity clock and the grace period

For a subject output, the node computes:

```
effective_last_active = max(coin_creation_height,
                           demurrage_activation_height,
                           latest_attestation_height)
inactive_blocks = spend_height - effective_last_active
```

If `inactive_blocks` $\leq$ `DEMURRAGE_GRACE_BLOCKS` (243,000 blocks = 6 months), the output is **exempt**, full value, no decay ("young" if freshly created/moved, "attested" if kept alive by a qualifying attestation). Creating, moving, or receiving an output resets its clock. A qualifying attestation resets the clock only for an eligible direct/tiered v16 key.

6.3 The decay curve

Past the grace period, value decays **quadratically** to zero over an 18-month window (`DemurrageRemainingPpm`, `src/consensus/demurrage.cpp`):

```

grace_blocks = 243,000          (6 months)
zero_blocks  = 972,000          (24 months)
decay_window = 729,000          (18 months)

elapsed      = inactive_blocks - grace_blocks
t            = (elapsed / 729,000) x 1,000,000    (parts per million)
remaining    = 1,000,000 - (t² / 1,000,000)        (ppm of value retained)

```

The value burned when such a coin is spent is **nominal** - **effective**. Consensus recognizes only the effective value as input principal. The transaction fee is then **effective inputs** - **outputs**, so the burned remainder is not a fee and is never paid to the block producer. A coin stake is governed by the same rule: only effective principal is returned, and its reward remains limited to the ordinary PoS subsidy plus explicit fees.

Decay table (a subject output with no qualifying attestation, spend/recreation, or successful coin stake):

Months inactive	Value retained
≤ 6 (grace)	100.0%
9	97.2%
12	88.9%
15	75.0%
18	55.6%
21	30.6%
24	0.0% (locked)

The quadratic curve begins after the six-month grace period and reaches zero effective value at 24 months of inactivity. The table gives intermediate retained-value examples.

6.4 Keeping eligible holdings at 100%

There are three ways to keep a quantum holding at full value. Which one applies depends on the output type and wallet state:

- **Wallet-assisted liveness attestation.** A demurrage attestation is a zero-value, fee-only transaction carrying an ML-DSA signature (`senddemurrageattestation`) that resets an eligible direct or tiered v16 key's clock. An attestation is valid for **6 months** (`DEMURRAGE_ATTEST_VALIDITY_BLOCKS` = 243,000), and the wallet attempts one at the **3-month** mark (`DEMURRAGE_AUTO_ATTEST_BLOCKS` = 121,500) only while staking is enabled, the private-key wallet is normally unlocked rather than staking-only, and a safe spendable fee input is available. Capacity, construction, or broadcast failures can defer an attempt; merely leaving a wallet online is not a guarantee.

- **Active cold staking.** Cold-stake (v14) outputs remain subject to demurrage. Each successful coin stake realizes any accrued burn, returns only effective principal plus the ordinary reward, recreates the output, and resets its activity clock. Delegation by itself is not an exemption.
- **Any ordinary use.** Moving, consolidating, or spending resets the clock as a side effect.

The `getdemurragewalletinfo` RPC reports, per output, the nominal amount, the current effective (post-decay) amount, the value that would be burned if spent now, whether an attestation is due, and whether the output is locked. `sweepdemurragedecay` spends outputs that are decaying but still have positive effective value, realizes the burn, and moves the remainder minus the explicit fee to a fresh quantum address. Outputs at zero effective value are permanently locked and are skipped. The GUI surfaces the same information and can request an attestation for an eligible selected address; normal unlock, key, fee, and broadcast requirements still apply.

Qualifying activity before decay preserves effective principal. Realized decay is burned when an output is spent.

7. Quantum Staking: Tiered, Cold, and Pooled

V4 supports tiered self-staking and cold staking with owner/staker key separation. A successful spend and recreation refreshes the relevant activity clock.

7.1 Tiered self-staking

A holder can bond quantum coins into a **tiered self-staking** output that encodes a lock schedule directly in the witness program (`QuantumStakeTierProgram`, `src/consensus/quantum_witness.h`):

- **State machine:** `BONDED` -> (initiate unbonding) -> `UNBONDING` until `unlock_height` -> spendable.
- **Unbonding period** is chosen by the holder (in blocks; at 64 s/block, e.g. 40,500 blocks = 30 days) and is stored in the program itself, so the lock is consensus-visible.
- **RPCs:** `fundquantumstakeaddress` (bond), `withdrawquantumstakeaddress` (unbond or withdraw matured funds, the call is state-aware), `getquantumstakeaddressinfo`, `listquantumstakeoutputs`.

The pool logic below ranks eligible locks by their encoded lock terms.

7.2 Cold staking: separate the owner key from the staking key

Cold staking (witness v14) splits control into an **owner key** (can withdraw the principal) and a **staker key** (can only mint blocks with the coins), hashed together under the domain tag "*Quantum Quasar Cold Stake v2*" (`src/consensus/quantum_witness.cpp`). This lets a holder keep their principal in cold storage while a hot node, their own or a trusted operator's, stakes on their behalf.

- The delegated principal is always **owner-controlled**; the operator can never move it.
- Cold-stake outputs are subject to the same inactivity schedule. A successful coin stake returns effective principal plus reward and creates a fresh output, but delegation alone does not freeze the clock.
- **RPCs:** `fundquantumcoldstakeaddress`, `withdrawquantumcoldstakeaddress`, `getquantumcoldstakebalance`.

7.3 Operator bonds and the per-pool cap

Operators who stake on behalf of others post a **30-day operator bond** (40,500 blocks, `src/wallet/rpc/staking.cpp`; pool logic in `src/node/quantum_pool.h`), which registers a verified commitment. A **wallet/policy per-pool cap of 20%** (`QUANTUM_POOL_CAP_BPS = 2000`) steers new delegations when under-cap operators are available. The cap is wallet and delegation policy, not a consensus rule; it does not prevent an operator from exceeding that share.

- **RPCs:** `fundquantumoperatorbond`, `withdrawquantumoperatorbond`, `getquantumoperatorbondinfo`, `getwalletquantumpoolinfo` (verified value, share in basis points, valid/invalid claim counts, over-cap flag per operator).

7.4 Autonomous redelegation

An unlocked owner wallet can use the redelegation engine (`src/wallet/redelegation.h`) to move an eligible delegation after its current operator has produced no observed wins for the policy interval and a meaningfully better verified target is available. A pool exceeding the policy cap does not itself trigger redelegation.

- **Trigger:** `6 x expected_interval_blocks`, clamped to **300-4,050 blocks**, followed by deterministic per-delegation jitter of up to **1,350 blocks**.
- **Dampers:** attempts and successes have a **1,350-block** rate limit. A separate **1,350-block** probation period runs from the current delegation's activation height; it is not a probation period created by a failed attempt.
- **Ranking:** verified non-current operators are ranked first by observed liveness and then by smaller pool share. Over-cap targets are removed when an under-cap alternative exists. If every verified candidate is over the cap, the bootstrap fallback permits those candidates and reports the projection.
- **Execution prerequisites:** automatic mode must be enabled, the owner wallet must be normally unlocked with private keys, and the delegation must be safe and owner-spendable. A missing target or transaction failure leaves the delegation unchanged.
- **RPCs:** `getquantumredelegationinfo` (dry-run/status), `redelegatquantumcoldstake` (manual, with cap enforcement and a dry-run mode).

The policy can reduce maintenance for an unlocked owner wallet, but it is not a consensus guarantee of liveness or pool distribution and does not remove the need to monitor failed or deferred attempts.

8. Reserved v15 EUTXO Design and RGB

V4 includes an operational RGB commitment path and retains a reserved EUTXO encoding for inspection and future design work. Those two states must not be treated as equally enabled.

8.1 EUTXO witness v15 is frozen in v30.1.1

The reserved **EUTXO-shaped output** (witness v15, `src/addresstype.h`) commits to a *datum* and *validator script* as `SHA256("Quantum Quasar EUTXO v1" || SHA256(datum) || SHA256(validator))` (`src/script/solver.cpp`). That commitment does not authenticate a quantum owner. A validator-only spend design would therefore permit value movement without the ML-DSA ownership guarantee required by the migration.

v30.1.1 sets `EUTXO_ENABLED` to false. Wallet and raw-transaction construction reject v15 outputs, the `createeutxospend` and `createeutxotransition` RPCs intentionally return a disabled error, and

post-Gold-Rush consensus rejects both v15 funding and v15 spends. `decodeutxospend` and `verifyutxospend` remain inspection tools only; `importeutxostate` and `listeutxostates` only persist or report metadata and cannot make a v15 output spendable. **Do not send BLK to a witness-v15 address in v30.1.1.**

8.2 RGB, client-side fixed-supply assets

An **RGB commitment** anchors a client-side-validated asset state transition in a zero-value OP_RETURN output (`OP_RETURN <RGB1> <32-byte state hash>, src/script/solver.cpp`). Asset data remains off-chain; clients validate it against the on-chain commitment chain. The containing transaction still pays its applicable fee.

- **Tooling:** `createrrgbtransfer`, `acceptrrgbconsignment`, `exportrrgbconsignment`, `importrrgbcontract`, `importrrgbassignment`, `listrrgbassets`, plus raw `decodergbcommitment` / `verifyrrgbconsignment`.

9. Wallet and RPC Reference

The following RPCs are part of the V4 wallet and node interface. Their availability depends on the build, loaded wallet, network phase, and RPC requirements described below.

9.1 Chain and schedule

RPC	Purpose
<code>getquantumquasarinfo</code>	V4 phase, activation/Gold-Rush/deadline times, next-block phase
<code>getgoldrushstate</code>	Chain-level Gold Rush (Shadow Network) state
<code>getgoldrushinfo</code>	Gold Rush pools (PoS/PoW amounts), solver counts, wallet qualification
<code>getcirculatingssupply</code>	Demurrage-adjusted circulating supply; guarded full scan with explicit one-call consent outside its reviewed envelope
<code>getshadowresourceinfo</code>	Inspect optional supply-scan qualification, bounds, warnings, and progress
<code>abortcirculatingssupplyscan</code>	Request cooperative cancellation of the active full-supply scan
<code>getquantumpoolinfo</code>	Non-consensus quantum cold-stake pool registry

9.2 Staking, Gold Rush, and mining

RPC	Purpose
<code>getstakinginfo</code> / <code>staking</code>	Read / start-stop legacy PoS staking
<code>reservebalance</code>	Reserve coins from staking/spending
<code>getstakingdonationinfo</code> / <code>setstakingdonation</code>	Inspect the retired legacy facility / confirm it remains disabled
<code>getqqdevelopmentdonationinfo</code> / <code>setqqdevelopmentdonation</code>	Review and durably record fresh wallet-scoped consent for the exact direct-quantum development recipient
<code>checkkernel</code>	Test whether an input is a valid PoS kernel now
<code>sendshadowsignal</code>	Broadcast a QQSIGNAL for a recent PoS solve (Gold Rush PoS credit)
<code>sendshadowpowclaim</code>	Grind and submit a QQSPROOF Argon2id PoW claim

RPC	Purpose
<code>setpowmining / getpowmininginfo</code>	Control / inspect the in-process Argon2id miner
<code>createshadowpowclaimresolution</code>	Compatibility preview/sign surface; after explicit acknowledgement it returns signed resolution bytes but does not itself broadcast them
<code>getpowclaimrecoveryinfo</code>	Inspect the wallet-scoped recovery choice and current component gate without creating, signing, or broadcasting
<code>setpowclaimrecovery</code>	Record unset, pause-and-ask, or explicitly bounded automatic recovery policy; never starts mining or creates a transaction
<code>optimizeutxoset</code>	Restructure eligible wallet outputs for staking under the RPC's fee and safety rules

The Issue #37 release's additional manual and bulk wrappers, when present in a build, use the same component engine and preserve separate preview, sign-and-persist, and commit-and-broadcast authority. Use that build's RPC help for its exact public command names and arguments; do not infer commit authority from a signed draft.

9.3 Quantum addresses and migration

RPC	Purpose
<code>getnewquantumaddress / listquantumaddresses</code>	Create / list wallet-backed ML-DSA migration addresses
<code>createquantumkey</code>	Fail-closed deprecated stub; v30.1.1 never generates unstored raw key material
<code>createquantummigrationaddresses</code>	Encode a migration commitment from an existing ML-DSA public key
<code>dumpquantumkey</code>	Expert-only export from the selected normally unlocked wallet; disabled unless <code>-allowunsafequantumkeyrpc</code> ; staking-only unlock is rejected
<code>importquantumkey</code>	Import an ML-DSA key into the selected wallet
<code>migratetoquantum</code>	Sweep legacy coins into a quantum migration address
<code>migrategoldrushrewards</code>	Move Gold Rush reward outputs to a fresh quantum address
<code>getmigrationstatus</code>	Migration progress, eligible legacy amount, deadline countdown, advice

v30.1.0 exposed raw unstored key generation through `createquantumkey`. v30.1.1 retains that RPC name only as a fail-closed migration stub. New keys are created through wallet-backed RPCs and must be covered by a verified backup. `-allowunsafequantumkeyrpc` enables only `dumpquantumkey`; it does not make the node offline or restrict RPC access, so exports belong in a separately isolated offline process.

9.4 Quantum staking, cold staking, operator bonds

RPC	Purpose
<code>getnewquantumstakeaddress</code>	New tiered self-staking address
<code>fundquantumstakeaddress / withdrawquantumstakeaddress</code>	Bond / unbond-withdraw tiered stake
<code>getquantumstakeaddressinfo / listquantumstakeoutputs</code>	Inspect tiered stake state and outputs
<code>getnewquantumcoldstakingaddress / createcoldstakingaddress</code>	New cold-stake delegation address

RPC	Purpose
fundquantumcoldstakeaddress / withdrawquantumcoldstakeaddress	Fund / unbond-withdraw a delegation
getquantumcoldstakebalance / listquantumcoldstakingdelegations	Inspect cold-stake holdings
importquantumcoldstakingdelegation	Import delegation metadata
fundquantumoperatorbond / withdrawquantumoperatorbond / getquantumoperatorbondinfo	Operate a 30-day operator bond
getwalletquantumpoolinfo	Verified operator registry (value, share bps, claims, over-cap)
getquantumredelegationinfo / redelegatquantumcoldstake	Inspect / execute redelegation (dry-run supported)

9.5 Demurrage

RPC	Purpose
senddemurrageattestation	Send a liveness attestation for an eligible wallet-backed direct/tiered v16 key; cold-stake outputs cannot be attested
getdemurragewalletinfo	Per-output decay state, effective value, attestation-due flag
sweepdemurragedecay	Realize decay on still-spendable outputs and move the effective remainder

9.6 EUTXO inspection and RGB

RPC	Purpose
createeutxospend / createeutxotransition	Disabled in v30.1.1; always return the v15 ownership-authorization error
decodeeutxospend / verifyeutxospend	Inspect candidate or known v15-shaped records; never authorize or enable a spend
importeutxostate / listeutxostates	Persist / list inspection-only EUTXO metadata; does not make an output spendable
creatergbtransfer / acceptribbconsignment / exportrgbconsignment	RGB transfer lifecycle
importrgbcontract / importrgbassignment / listrgbassets	RGB asset management
decodergbcommitment / verifyrgbconsignment	Raw RGB inspection

9.7 Wallet maintenance

RPC	Purpose
deladdressbook	Remove a sending-address book entry (CLI parity with the GUI)
burn / burnwallet	Provably destroy specific coins / the whole wallet balance

9.8 The GUI

The Qt wallet adds two dedicated pages:

- **Staking & Mining**, one place for PoS staking, Gold Rush status, the in-process PoW miner, quantum migration, tiered/cold staking, operator bonds, demurrage, RGB, and inspection-only EUTXO metadata. Detail panels load on demand behind a **Refresh details** button, reducing work when the tab first opens.
- **Account**, a per-family (Legacy / Quantum / Cold-stake / EUTXO) breakdown of every output, its state (bonded / unbonding / withdrawable), and demurrage exposure, with CSV export. An EUTXO row is a frozen-output warning and inspection surface, not a funding or spend workflow.

The **Unlock Wallet** dialog offers two explicit, mutually-exclusive modes: **For Legacy Staking Only** (mint PoS blocks, no spending or quantum actions) and **Legacy and Quantum Staking** (full unlock, required for any quantum, Gold Rush, migration, or cold-staking transaction). Automatic demurrage-attestation attempts additionally require staking to be enabled and a safe spendable fee input.

In v30.1.1 through v30.1.3, the Staking & Mining dashboard reports a quarantined claim and directs advanced operators to the compatibility preview/sign RPC in the Qt debug console; those releases do not provide a one-click conflict broadcast.

The later Issue #37 release adds **Review claim recovery...** and a seventh wallet-scoped optional-automation control. The manual dialog and headless wrappers consume the same pinned component plan. They preserve separate preview, sign-and-persist, and commit-and-broadcast decisions and show fees, sibling conflicts, descendants, stale depth, and both confirmation outcomes. The automatic control is off until the operator records positive fee, batch, rolling-budget, rate, and staleness limits. Neither path guarantees peer acceptance or confirmation, unlocks the wallet, or turns mining on.

10. Worked Wallet Examples

Example A, A legacy holder preparing to migrate

Alice holds 250,000 BLK in eligible legacy outputs and plans to migrate.

1. **Before V4:** she verifies wallet access and preserves a current backup.
2. **During Gold Rush:** she creates and backs up a wallet-backed quantum address. She can dry-run migration planning, but does not fund it yet.
3. **During Migration, beginning at height 6,193,000:** she runs `migratetoquantum` with an existing backed-up wallet-owned address or explicitly authorizes a new key. She checks the transaction outcome and **backs up her wallet again** after creating a key because ML-DSA keys are not in the seed.
4. **Optional:** she runs `fundquantumcoldstakeaddress` to delegate to a cold-staking operator (or her own hot node). Her principal stays owner-controlled and can earn staking rewards. Successful coinstakes refresh the output's activity clock.
5. **Afterward:** if she leaves coins in a direct quantum address, her wallet can attempt an attestation at 3 months while staking is enabled, normally unlocked, and funded by a safe fee input. She still monitors the reported state because an attempt can be deferred. If she delegates, she monitors successful staking or moves the output before prolonged inactivity; delegation alone is not an exemption and cold-stake outputs cannot be attested.

Alice prepares an address and backup during Gold Rush, runs migration during Migration, and can optionally add a staking workflow.

Example B, The active staker during Gold Rush

Bob holds 40,000 BLK (above the 10,000 whitelist threshold) and runs a node.

1. His account is captured in the whitelist snapshot at height 5,945,000.
2. During Gold Rush he keeps staking. After a qualifying solve, he can submit a QCSIGNAL; an eligible, normally unlocked wallet can attempt this automatically only when optional QCSIGNAL automation is enabled and its payout prerequisites are met. A confirmed signal remains active for the 14-day window. PoS pool credit also depends on the applicable solver and signal rules.
3. If he wants to use the PoW lane, he enables `setpowmining`. The Argon2id puzzle uses a 1 MiB memory parameter; a valid `sendshadowpowclaim` proof can qualify for the PoW pool under the claim rules.
4. He migrates during the 18-month Migration phase, before Final height 6,922,000.

Bob still needs eligible solves, confirmed signals or claims, and applicable fees; operating the node alone does not guarantee a reward.

Example C, The forgotten wallet

Carol migrated to a quantum address during Migration, then left her wallet offline.

- For **six months** after demurrage activates: no effect. 100% retained.
- At **12 months** of total inactivity: 88.9% retained.
- If she returns before the terminal **24-month** boundary and submits a valid liveness attestation, the clock resets before decay is realized. If she spends first, the spend burns the accrued difference and moves only the effective remainder.
- If she reaches a full **24 months** without qualifying activity, the output reaches zero effective value and becomes permanently unspendable. No miner or staker receives it.

Example D, Running a staking pool

Dan wants to stake on behalf of others.

1. He posts a 30-day operator bond (`fundquantumoperatorbond`), registering a verified commitment.
2. Delegators send him cold-stake delegations; their principals remain theirs. The 20% wallet-policy cap steers new delegations toward under-cap alternatives when they exist, but it does not prevent an operator from exceeding that share.
3. If Dan's node stops producing, a normally unlocked owner wallet can redelegate an eligible, owner-spendable delegation after the clamped zero-win trigger, rate limits, probation, and jitter, but only if a meaningfully better verified target is available. The owner still monitors failures and prolonged inactivity.

11. Economic Incentives and Participation Conditions

V4 defines reward eligibility, inactivity treatment, and legacy spending by phase. These rules change the conditions for earning Gold Rush rewards and retaining the effective value of eligible quantum holdings.

- **Gold Rush** front-loads a large, deterministic reward (up to 51,437,700 BLK) at the transition. Its PoS and PoW pools pay qualifying staking and mining claims under separate eligibility rules.
- **Demurrage** reduces the effective value of eligible quantum principal after the inactivity threshold. Realized decay is burned, not transferred to stakers. Active stakers receive the ordinary subsidy and explicit transaction fees under the applicable rules.
- **Final Lockout** rejects legacy ECDSA spending after the published height. Eligible legacy outputs must be migrated during Migration to remain spendable through an ML-DSA-protected path.
- **Tiered, cold, and pooled staking** lower the barrier to participation: cold delegation lets a holder participate without exposing the owner key on a hot node, and conditional owner-wallet redelegation can steer stale delegations toward better verified operators.

Participation still depends on wallet availability, eligible outputs, successful solves or claims, and applicable fees. The protocol rules do not guarantee individual rewards or a market outcome.

12. Security Considerations

- **Quantum resistance is opt-in-by-deadline, not instant.** During Gold Rush and Migration (approximately 720 target days), legacy ECDSA coins remain spendable and therefore quantum-exposed. Holders can prepare addresses and backups during Gold Rush, then should migrate during the 540-day Migration phase. At Final Lockout the spendable path becomes quantum-only. The published height schedule bounds the period for legacy spending and migration.
- **ML-DSA keys are outside the HD seed. Back up the wallet after every new quantum address.** A seed phrase alone does not recover ML-DSA-protected funds. The wallet enforces "key stored before funds move" and warns at each step, but the backup responsibility is the user's.
- **Attestations and consensus determinism.** Demurrage attestations are ML-DSA-signed and bound to the first input's outpoint as a replay anchor, and are validated in-consensus, so every node computes identical effective values and no attestation can be replayed onto a different coin.
- **Witness v15 is not an enabled smart-contract path.** Its datum/validator commitment lacks ML-DSA owner authorization. v30.1.1 disables supported v15 construction in every phase, and consensus rejects v15 outputs and spends from Migration onward. Its decode, verification, and wallet-metadata surfaces are inspection-only.
- **A mempool departure is not safe claim abandonment.** A peer can retain a base-valid **QQSPR00F** after the local node removes it. v30.1.1 therefore reserves the exact input until the original or a confirmed conflict resolves it. Guided conflict construction is dry-run by default, requires explicit acknowledgement to sign, never broadcasts, and does not promise a no-loss result.
- **Resource diagnostics are scoped, not consensus permissions.** Optional full-supply scans are single-flight, bounded, progress-reporting, and cooperatively cancellable. One-call operator consent cannot bypass critical record/seek, integrity, storage, snapshot, overflow, shutdown, or cancellation protections. A

successful qualification is fixed-height and host-scoped and reports `universal_consensus_bound=false`.

- **Live witness evidence is optional post-release qualification.** When run, the exact-source gate binds the final daemon and CLI to a fresh connected-tip mainnet UTXO MuHash, complete value-bearing witness-v2-through-v16 inventory, and same-tip shadow reconciliation. It requires either no bridge-review outpoints or an approved disposition for every such outpoint. Missing runners, capture paths, maturity, or evidence do not block publication and must not be represented as a successful live qualification.
- **Consensus compatibility.** Mainnet's whitelist height (5,945,000), Gold Rush boundaries (5,950,000 through 6,192,999), Migration boundaries (6,193,000 through 6,921,999), and Final Lockout height (6,922,000) are consensus rules. The retained timestamp anchors are nominal forecasts, not mainnet phase boundaries. Every node that wishes to remain on the same chain must use identical height values. Operators upgrading or building alternative clients must match them exactly to avoid a chain split.
- **The per-pool cap is policy, not consensus.** It cannot by itself prevent an operator from accumulating stake; it only steers default wallet behavior. Operator distribution also depends on delegators' choices.

Appendix A: Consensus Constant Reference

Values reflect the v30.1.1 release source. The nominal time anchors and durations descend from v30.1.0. v30.1.1 makes the mainnet lifecycle height-authoritative, starts demurrage automatically at Final Lockout, and adds the competing-claim boundary shown below.

Constant	Value	Meaning	Defined in
QUANTUM_QUASAR_MAINNET_V4_TIME	1783835299 (2026-07-12 05:48:19 UTC)	Nominal V4 time anchor; v30.1.1 mainnet lifecycle is height-authoritative	consensus/params.h
QUANTUM_QUASAR_GOLD_RUSH_SECONDS	15,552,000 (180 days)	Gold Rush duration	consensus/params.h
QUANTUM_QUASAR_MIGRATION_SECONDS	46,656,000 (540 days)	Migration window	consensus/params.h
Nominal final-lockout time	1846043299 (2028-07-01 05:48:19 UTC)	Non-authoritative time-schedule reference (V4 + 720 days)	derived
SHADOW_WHITELIST_HEIGHT	5,945,000	Balance snapshot height	shadow_schedule.cpp
SHADOW_WHITELIST_MIN_BALANCE	10,000 BLK	Whitelist eligibility threshold	shadow.h
SHADOW_REWARD_START_HEIGHT	5,950,000	Gold Rush rewards begin	shadow_schedule.cpp
MAINNET_SHADOW_COMPETING_CLAIMS_ACTIVATION_HEIGHT	5,993,200	Canonical competing-claim allocation begins	shadow.h
Consensus::Params::nShadowQQP4ActivationHeight	INT_MAX (disabled in v30.1.1)	Separately scheduled exact-input QQP4 activation; not a readiness-bit activation	consensus/params.h
SHADOW_GOLD_RUSH_BLOCKS	243,000 (180 days)	Gold Rush length	shadow_schedule.cpp
SHADOW_REWARD_END_HEIGHT	6,192,999	Gold Rush rewards end	shadow_schedule.cpp

Constant	Value	Meaning	Defined in
MAINNET_QUANTUM_MIGRATION_END_HEIGHT	6,921,999	Last height-authoritative Migration block	shadow.h
MAINNET_QUANTUM_FINAL_START_HEIGHT	6,922,000	Height-authoritative Final Lockout and automatic demurrage begin	shadow.h
SHADOW_PHASE1_END_HEIGHT	6,187,599	Halving phase ends	shadow_schedule.cpp
SHADOW_HALVING_INTERVAL_BLOCKS	43,200 (~30 days)	Reward-halving period	shadow_schedule.cpp
Phase-1 base reward	580 BLK/block, halving	Gold Rush emission (halving)	shadow.cpp
Phase-2 base reward	463 BLK/block, flat	Gold Rush emission (tail)	shadow.cpp
SHADOW_MAX_EMISSION	51,437,700 BLK	Gold Rush issuance cap	shadow.h
PoS / PoW split	50% / 50%	Per-block reward pool split	shadow.cpp
SHADOW_SOLVER_ACTIVITY_WINDOW	18,900 blocks (14 days)	Recent-PoS-solve window for signalling	shadow.h
Argon2id (time, mem, lanes)	1, 1024 KiB, 1	PoW puzzle parameters	shadow.cpp
DEMURRAGE_GRACE_BLOCKS	243,000 (6 months)	No-decay grace period	consensus/demurrage.h
DEMURRAGE_ZERO_BLOCKS	972,000 (24 months)	Full-decay point	consensus/demurrage.h
Decay window	729,000 (18 months)	Quadratic decay span	consensus/demurrage.cpp
DEMURRAGE_ATTEST_VALIDITY_BLOCKS	243,000 (6 months)	Attestation validity	consensus/demurrage.h
DEMURRAGE_AUTO_ATTEST_BLOCKS	121,500 (3 months)	Conditional wallet attempt threshold	consensus/demurrage.h
DEMURRAGE_BLOCKS_PER_MONTH	40,500	Block/month conversion	consensus/demurrage.h
ML-DSA public key	1,312 bytes	Quantum public key size	crypto/mldsa.h
ML-DSA signature	2,420 bytes	Quantum signature size	crypto/mldsa.h
QUANTUM_MIGRATION_PROGRAM_SIZE	32 bytes	Direct v16 program	consensus/quantum_witness.h
QUANTUM_TIERED_PROGRAM_SIZE	40 bytes	Tiered v14/v16 program	consensus/quantum_witness.h
EUTXO_PROGRAM_SIZE	32 bytes	Reserved v15 shape; funding and spending disabled in v30.1.1	addresstype.h
QUANTUM_COLDSTAKE_PROGRAM_SIZE	32 bytes	Direct v14 program	consensus/quantum_witness.h
QUANTUM_POOL_CAP_BPS	2000 (20%)	Per-pool delegation cap (policy)	node/quantum_pool.h
QuantumRedelegationPolicy::trigger_multiplier	6	Expected zero-win interval multiplier	wallet/redelegation.h
QuantumRedelegationPolicy::min_trigger_blocks	300 blocks	Minimum zero-win trigger	wallet/redelegation.h

Constant	Value	Meaning	Defined in
QuantumRedelegationPolicy::max_patience_blocks	4,050 blocks	Maximum zero-win trigger	wallet/redelegation.h
QuantumRedelegationPolicy::rate_limit_blocks	1,350 blocks	Attempt/success rate limit	wallet/redelegation.h
QuantumRedelegationPolicy::probation_blocks	1,350 blocks	Current-target activation probation	wallet/redelegation.h
QuantumRedelegationPolicy::stampede_jitter_blocks	1,350 blocks	Deterministic maximum jitter	wallet/redelegation.h
QuantumRedelegationPolicy::liveness_improvement_blocks	300 blocks	Required target liveness improvement	wallet/redelegation.h
Operator bond period	40,500 blocks (30 days)	Verified operator commitment	wallet/rpc/staking.cpp
Block time	64 seconds	Mainnet target spacing	kernel/chainparams.cpp
Bech32 HRP	blk	Mainnet address prefix	kernel/chainparams.cpp

Appendix B: Glossary

- **ML-DSA-44**, Module-Lattice Digital Signature Algorithm (FIPS 204), the post-quantum signature scheme used for all quantum spends and attestations.
- **Gold Rush**, the 180-day bonus-emission epoch that launches V4, paid to stakers and miners.
- **Whitelist**, the deterministic set of scripts holding $\geq 10,000$ BLK at height 5,945,000, eligible to earn Gold Rush PoS credits.
- **QQSIGNAL / QQSPROOF**, the OP_RETURN control transactions that claim Gold Rush PoS and PoW rewards respectively.
- **Migration**, moving legacy ECDSA coins into quantum (v16) outputs via [migratetoquantum](#).
- **Final Lockout**, mainnet height 6,922,000, after which legacy ECDSA spends are permanently rejected and demurrage begins automatically.
- **Demurrage**, the liveness mechanism by which inactive quantum outputs slowly lose effective value. Realized decay is burned, never paid to a miner or staker. Timely attestation or a spend/recreation refreshes activity; cold delegation alone does not.
- **Attestation**, a zero-value ML-DSA-signed transaction that resets an eligible direct or tiered v16 key's demurrage clock. Cold-stake outputs cannot be attested. The wallet can attempt attestations only when its staking, unlock, key, fee-input, and capacity prerequisites are satisfied.
- **Cold staking**, owner/staker key separation (witness v14) letting a holder delegate staking while retaining principal control; subject to the inactivity schedule.
- **Tiered staking**, self-staking with a consensus-visible bonding/unbonding lock schedule.
- **Operator bond**, a 30-day verified commitment posted by a staking-pool operator.
- **EUTXO**, a reserved witness-v15 datum/validator commitment shape. v30.1.1 freezes its funding and spending pending a quantum-authenticated ownership design.
- **RGB**, client-side-validated, fixed-supply asset commitments anchored on-chain.

This document describes the Blackcoin Quantum Quasar (Protocol V4) v30.1.1 release. All consensus boundaries are drawn from the v30.1.1 source. Blackcoin is free/open-source software under the MIT license.